LLM-Generated Reward Functions for RL-Based Robot Navigation

Bachelor Thesis — Progress Report

Mohamed

Computer Engineering, University of Paderborn

September 9, 2026

This report documents the design and implementation of a modular Reinforcement Learning framework in Python/PyTorch, the integration of Proximal Policy Optimisation (PPO) trained on a custom grid-world environment, and an analysis of the current performance ceiling and identified bottlenecks.

Contents

1 Overview and Motivation

The long-term objective of this thesis is to enable a ROSbot 2 Pro to navigate autonomously in unknown environments using a reward function generated on the fly by a Large Language Model (LLM). Before addressing the full robotic pipeline, a custom grid-world simulator was built from scratch to validate the core RL components in a controlled setting where ground truth is known.

This report covers:

- the MDP formulation and environment architecture,
- the state representation and reward function,
- the PPO implementation and training procedure,
- experimental results on two grid sizes,
- identified bottlenecks and the planned path forward.

2 Environment: MDP Formulation

2.1 Markov Decision Process

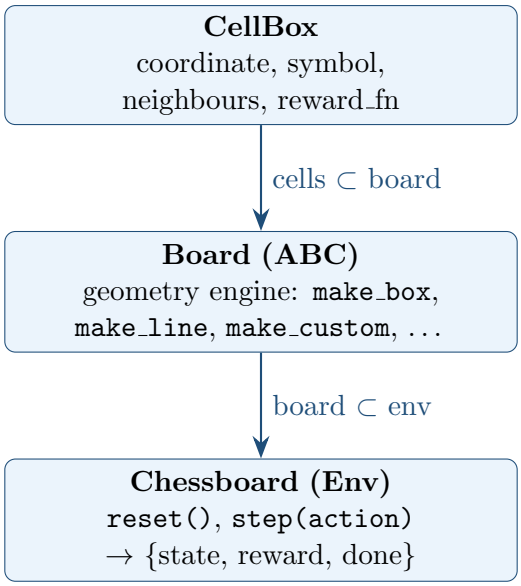
The problem is formalised as a finite, discrete-time MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ with the following components.

Table 1: MDP Components

Symbol	Name	Description
$\mathcal{S}$	State space	Set of grid cells; each cell s carries a coordinate (x, y), a symbol $\sigma \in \{\texttt{O}, \texttt{X}, \texttt{None}\}$, and a reference to its board.
$\mathcal{A}$	Action space	$\{\texttt{up}, \texttt{right}, \texttt{down}, \texttt{left}\}$ — four cardinal moves.
$\mathcal{P}$	Transition	Deterministic: taking action a in state s always produces the same s'; hitting a wall leaves the agent in s.
$\mathcal{R}$	Reward	See Section 4.
γ	Discount	0.99 (large grid); 0.90 (small grid).

2.2 Environment Architecture

The implementation separates geometry from RL semantics through a three-layer class hierarchy:



Key design principle: `Board` handles topology only; `Chessboard` handles RL concepts (transitions, rewards, episode termination). Swapping a 3×3 grid for the large board requires only changing the `Board` argument — no other code changes.

3 State Representation

3.1 Feature Vector $\phi(s)$

The state vector $\phi(s) \in \mathbb{R}^9$ encodes local perception and global navigation information.

Local perception (5 features). For each of the four cardinal directions, the neighbouring cell is encoded as:

$$\phi_{\text{dir}}(s) = \begin{cases} -1.0 & \text{trap adjacent in this direction} \\ 0.0 & \text{empty cell} \\ +1.0 & \text{goal adjacent in this direction} \\ +2.0 & \text{wall (no neighbour)} \end{cases}$$

The current cell is encoded using the same symbol mapping:

$$\phi_{\text{current}}(s) = \begin{cases} -1.0 & \text{current cell is a trap} \\ 0.0 & \text{current cell is empty} \\ +1.0 & \text{current cell is a goal} \end{cases}$$

Thus, the local perception features are:

$$[\phi_{\text{up}}, \phi_{\text{right}}, \phi_{\text{down}}, \phi_{\text{left}}, \phi_{\text{current}}]$$

Global navigation signals (4 features). Let g^* denote the nearest goal and t^* denote the nearest trap. The global features are defined as:

$$\phi_6 = \frac{d(s, g^*)}{d_{\max}} \qquad \text{(normalised distance to nearest goal)}$$

$$\phi_7 = \frac{g_x^* - s_x}{d_{\max}} \qquad \text{(signed } x\text{-offset toward nearest goal)}$$

$$\phi_8 = \frac{g_y^* - s_y}{d_{\max}} \qquad \text{(signed } y\text{-offset toward nearest goal)}$$

$$\phi_9 = \frac{d(s, t^*)}{d_{\max}} \qquad \text{(normalised distance to nearest trap)}$$

The Manhattan distance between two cells a and b is defined as:

$$d(a, b) = |a_x - b_x| + |a_y - b_y|$$

The normalisation factor is defined as:

$$d_{\max} = \max_x + \max_y$$

based on the maximum cell coordinates of the current board.
Therefore, the complete state representation is:

$$\boxed{\phi(s) = [\phi_{\text{up}}, \phi_{\text{right}}, \phi_{\text{down}}, \phi_{\text{left}}, \phi_{\text{current}}, \phi_{\text{dist-goal}}, \phi_{\text{goal-dx}}, \phi_{\text{goal-dy}}, \phi_{\text{dist-trap}}] \in \mathbb{R}^9}$$

3.2 Pseudo-code

Listing 1: State encoding $\phi(s)$

```
function state_to_vector(s):

    # Local perception
    for dir in [up, right, down, left]:

        nb = s.neighbours[dir]

        if nb is None:
            local[dir] = +2.0

        elif nb.symbol == X:
            local[dir] = -1.0

        elif nb.symbol == O:
            local[dir] = +1.0

        else:
            local[dir] = 0.0

    # Current cell
    current = encode(s.symbol)

```

```
    # Global navigation
    g* = nearest_goal(s)

    dist_goal =
        manhattan(s, g*) / d_max

    dx =
        (g*.x - s.x) / d_max

    dy =
        (g*.y - s.y) / d_max

    # Trap awareness
    t* = nearest_trap(s)

    dist_trap =
        manhattan(s, t*) / d_max

    return [
        local[up],
        local[right],
        local[down],
        local[left],
        current,
        dist_goal,
        dx,
        dy,
        dist_trap
    ]
```

“‘latex

4 Reward Function

The reward function defines the numerical feedback received by the agent after each environment transition. In the implemented environment, a transition is represented by

$$s_t \xrightarrow{a_t} s_{t+1},$$

and the corresponding reward is defined as

$$r_t = R(s_t, a_t, s_{t+1}).$$

Although the reward interface accepts the complete transition (s_t, a_t, s_{t+1}), the hand-crafted reward functions implemented in this work primarily depend on the resulting state s_{t+1}.

4.1 Hand-crafted Reward Functions

Three hand-crafted reward functions were implemented to investigate how different reward structures influence the learning behaviour of the agent.

4.1.1 Version 1: Baseline Reward

The baseline reward function assigns a positive reward for reaching a goal, a negative reward for entering a trap, and a constant penalty for every non-terminal transition:

$$R_1(s_t, a_t, s_{t+1}) = \begin{cases} +100 & \text{if } s_{t+1}.\sigma = \texttt{O} \quad \text{(goal reached)} \\ -100 & \text{if } s_{t+1}.\sigma = \texttt{X} \quad \text{(trap entered)} \\ -10 & \text{otherwise} \quad \text{(step penalty)} \end{cases}$$

This reward function provides a simple baseline consisting of terminal rewards and a constant step penalty.

4.1.2 Version 2: Strong Terminal Rewards

The second reward configuration increases the magnitude of terminal rewards and penalties while reducing the penalty associated with normal movement:

$$R_2(s_t, a_t, s_{t+1}) = \begin{cases} +500 & \text{if } s_{t+1}.\sigma = \texttt{O} \quad \text{(goal reached)} \\ -500 & \text{if } s_{t+1}.\sigma = \texttt{X} \quad \text{(trap entered)} \\ -1 & \text{otherwise} \quad \text{(step penalty)} \end{cases}$$

Compared to the baseline, this configuration places greater emphasis on the terminal outcome while making intermediate movement relatively less costly.

4.1.3 Version 3: Distance-Based Reward Shaping

The third reward function introduces a shaping signal based on the distance between the resulting state and the nearest goal.

Let G denote the set of all goal cells:

$$G = \{g \in \mathcal{B} \mid g.\sigma = \texttt{O}\}.$$

The Manhattan distance between the resulting state s_{t+1} and a goal g is defined as

$$d(s_{t+1}, g) = \left|x_{s_{t+1}} - x_g\right| + \left|y_{s_{t+1}} - y_g\right|.$$

The distance to the nearest goal is therefore

$$d(s_{t+1}, G) = \min_{g \in G} d(s_{t+1}, g).$$

The resulting reward function is

$$R_3(s_t, a_t, s_{t+1}) = \begin{cases} +100 & \text{if } s_{t+1}.\sigma = \texttt{O} \quad \text{(goal reached)} \\ -100 & \text{if } s_{t+1}.\sigma = \texttt{X} \quad \text{(trap entered)} \\ -d(s_{t+1}, G) & \text{if } G \neq \emptyset \quad \text{(distance to nearest goal)} \\ -10 & \text{if } G = \emptyset \quad \text{(no goal available)} \end{cases}$$

This configuration provides denser feedback than the previous reward functions by assigning different rewards to non-terminal states according to their distance from the nearest goal.

4.2 Reward Function Integration

The reward function is implemented as a configurable component of the `Chessboard` environment. The environment accepts an optional reward function during initialization:

Listing 2: Configurable reward function interface

```
def __init__(
    self,
    board,
    actions,
    start,
    reward_fn=None
):
```

If no custom reward function is provided, the environment uses its default reward function:

Listing 3: Default reward assignment

```
if reward_fn is None:
    self.reward_fn = self.default_reward
else:
    self.reward_fn = reward_fn
```

During each environment transition, the reward is calculated by passing the current state, selected action, and resulting state to the assigned reward function:

Listing 4: Reward evaluation during a transition

```
reward = float(
    self.reward_fn(state, action, next_state)
)
```

Therefore, the reward mechanism can be represented as

$$\boxed{r_t = \texttt{reward\_fn}(s_t, a_t, s_{t+1})}$$

This architecture separates the reward definition from the transition mechanism. Consequently, different reward functions can be evaluated without modifying the board topology, state transitions, or terminal conditions.

4.3 Reward Function Configurations

The configurable reward interface allows different reward functions to be assigned to different instances of the same environment:

Listing 5: Reward function injection

```
# Default reward function
env1 = Chessboard(
    board=board_v1,
    actions=Actions,
    start=s1
)

# Strong terminal reward function
env2 = Chessboard(
    board=board_v2,
    actions=Actions,
    start=s1,
    reward_fn=reward_v2
)

# Distance-based reward shaping
env3 = Chessboard(
    board=claude_board,
    actions=Actions,
    start=s1,
    reward_fn=reward_v3
)
```

Conceptually, the environment architecture can therefore be expressed as

$$\boxed{\text{Chessboard} + R_i(s_t, a_t, s_{t+1})}$$

where R_i represents a selected reward function configuration.

This design also provides an integration point for future LLM-generated reward functions:

$$\text{Task Description} \rightarrow \text{LLM} \rightarrow \text{Reward DSL} \rightarrow R_{\text{LLM}} \rightarrow \text{Chessboard Environment}.$$

The generated reward function only needs to conform to the interface

$$R(s_t, a_t, s_{t+1}) \rightarrow r_t.$$

4.4 Reward Scaling

Reward scaling is applied inside the training loop to maintain numerically stable learning targets. The scaling operation is defined as

$$\tilde{r}_t = \frac{r_t}{c},$$

where c is a predefined scaling constant.

The scaled reward $\tilde{r}_t$ is subsequently used by the learning algorithm for return and advantage estimation.

Current limitation: constant step penalties

The baseline reward function assigns the same penalty to every non-terminal transition. Consequently, longer paths receive a larger accumulated penalty regardless of whether the additional steps are necessary to avoid traps.
For example, an agent may need to temporarily move away from the goal in order to follow a safer path. The reward function does not directly distinguish such a necessary detour from inefficient movement. Consequently, the agent must learn this distinction from delayed terminal feedback.
A future extension may introduce a progress-based shaping component that rewards movement toward the objective while preserving the importance of avoiding traps.

‘‘‘

5 PPO Implementation

5.1 Algorithm Overview

The PPO algorithm [1] is implemented from scratch in PyTorch following the canonical formulation.

Algorithm 1 PPO Training Loop

for each environment e in training set **do**
  for each episode **do**
    Reset $s_0 \leftarrow$ `env.reset()`
    for $t = 1, \ldots, T$ **do**
      $a_t, \log \pi_\theta(a_t|s_t), V_\theta(s_t) \leftarrow$ `agent.act`(s_t)
      $s_{t+1}, r_t, d_t \leftarrow$ `env.step`(a_t)
      $\delta_t \leftarrow \begin{cases} r_t - V_\theta(s_t) & d_t = \texttt{True} \\ r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t) & \text{otherwise} \end{cases}$
      Store $(s_t, a_t, \delta_t, \log \pi_\theta, V_\theta(s_t), d_t)$
      if rollout full **or** d_t **then**
        Compute $\hat{A}_t^{\text{GAE}}$ (backward pass)
        $\hat{y}_t \leftarrow \hat{A}_t + V_\theta(s_t)$
        for $k = 1, \ldots, K$ epochs **do**
          Minimise $\mathcal{L}^{\text{PPO}}$ via Adam
        end for
        Clear rollout buffer
      end if
    end for
  end for
end for

5.2 Generalised Advantage Estimation

Advantages are computed using GAE with episode-boundary resets:

$$\hat{A}_t^{\text{GAE}} = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}, \qquad \lambda = 0.95$$

The backward recursion resets $\hat{A}_{t+1} = 0$ whenever $d_t = \texttt{True}$, preventing advantage leakage across episode boundaries.

5.3 Loss Function

The total PPO loss combines three terms:

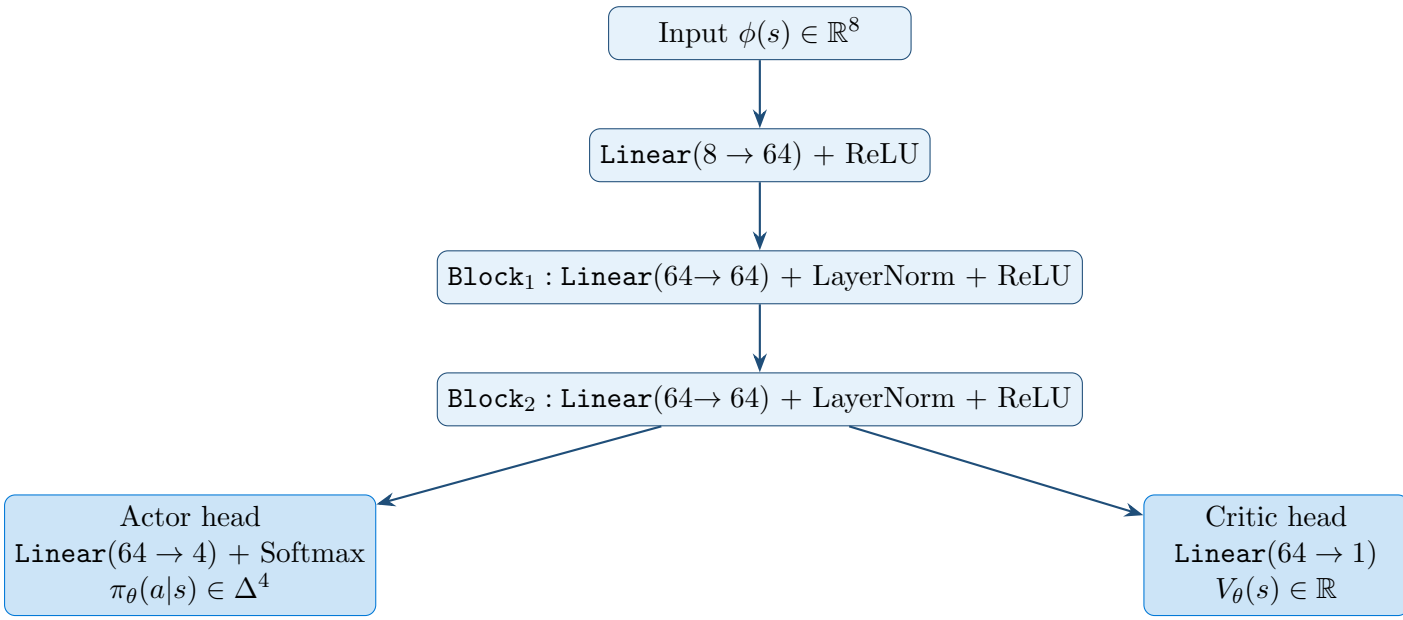
$$\mathcal{L}^{\text{PPO}} = \underbrace{\mathcal{L}^{\text{CLIP}}}_{\text{actor}} + c_v \underbrace{\mathcal{L}^{\text{VF}}}_{\text{critic}} - c_e \underbrace{\mathcal{H}[\pi]}_{\text{entropy}}$$

$$\mathcal{L}^{\text{CLIP}} = -\mathbb{E}_t\Big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t\big)\Big], \quad \rho_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

$$\mathcal{L}^{\text{VF}} = \mathbb{E}_t\big[(V_\theta(s_t) - \hat{y}_t)^2\big]$$

$$\mathcal{H}[\pi] = -\mathbb{E}_t\big[\textstyle\sum_a \pi_\theta(a|s_t) \log \pi_\theta(a|s_t)\big]$$

5.4 Actor-Critic Network Architecture



The shared body learns a common state representation; the two heads are trained by separate gradient clipping budgets ($\|\nabla\| \leq 0.5$ each) to prevent the critic from starving the actor.

Table 2: Hyperparameters for both agents

Parameter	`agent` (small grid)	`agent_carter` (large grid)
Hidden dim	64	64
Residual blocks	2	4
Clip ε	0.20	0.20
γ	0.90	0.99
λ (GAE)	0.95	0.95
c_v (critic weight)	0.50	0.70
c_e (entropy weight)	0.025	0.05
Epochs per rollout K	6	10
Rollout length	256	640
Learning rate	3×10^{-4}	3×10^{-4}
Optimiser	Adam	Adam

5.5 Hyperparameters

6 Training Procedure

6.1 Environment Randomisation

To prevent the agent from memorising fixed layouts, each training episode uses a freshly randomised board. Goals and traps are placed uniformly at random on free cells (never on the designated start cell).

Listing 6: Environment randomisation

```
function randomize_board(env, n_goals, n_traps):
    clear all symbols
    candidates = env.states \ {env.start}   # exclude start
    shuffle(candidates)
    assign O to candidates[0 : n_goals]
    assign X to candidates[n_goals : n_goals + n_traps]
```

6.2 Training Distribution

Two training regimes were used:

Table 3: Training configurations

Agent	Board	Goals	Traps	Environments
`agent`	3×3 (9 cells)	1	1	50
`agent_carter`	Global ($6 \times 9 = 54$ cells)	3	10	100

Environments are shuffled before training so the agent sees mixed difficulty rather than an ordered curriculum. Each training environment ran for 600 episodes with up to 5 000 steps per episode.

6.3 Gradient Balance Monitoring

A custom diagnostic (`debug_learning_balance`) tracks the ratio of actor to critic gradient norms after each update. The target range is $[0.7, 1.3]$.

Table 4: Observed gradient balance

Agent	Actor norm	Critic norm	Ratio
`agent`	0.033	0.034	0.97 ✓
`agent_carter`	0.066	0.084	0.79 ✓

7 Experimental Results

7.1 Small Grid (3×3)

`agent` was tested on 10 randomised environments (15 episodes each, max 500 steps).

Table 5: Small-grid test results (10 environments, 15 episodes each)

Metric	Value
Average success rate	34%
Best environment	12/15 (80%)
Worst environment	0/15 (0%)
Dominant failure mode	Timeout (path too long)

7.2 Large Grid (6×9, 54 cells)

`agent_carter` was tested on 10 environments (10 episodes each, max 50 steps).

Table 6: Large-grid test results (10 environments, 10 episodes each)

Metric	Value
Average success rate	73%
Perfect environments (10/10)	5 out of 10
Failed environments (0/10)	1 out of 10
Dominant failure mode	Trap collision (blocked path)
Average steps to goal (successes)	4–11

Key observation

When the direct path to the goal is unobstructed, `agent_carter` achieves 10/10 success consistently and reaches the goal in near-optimal steps. Failure occurs exclusively when traps physically block the shortest Manhattan path, forcing a detour the agent has not learned to take.

8 Bottleneck Analysis

8.1 Root Cause: Local Minimum in Navigation

The failure pattern is not overfitting. Overfitting would manifest as high performance on training environments and low performance on unseen environments of the same type. What is observed instead is:

- **Success** when the direct path (lowest Manhattan distance) is clear.
- **Failure** when a trap sits on the direct path, regardless of whether the environment was seen during training.

This is the classic *local minimum problem* in potential-field navigation: the agent descends the distance-to-goal gradient and cannot escape a local minimum created by a nearby obstacle.

8.2 Contributing Factors (Ranked by Impact)

Table 7: Bottleneck ranking

Rank	Factor	Explanation
1	State representation	ϕ_7, ϕ_8 (dx, dy) point toward the goal without encoding path quality per direction. The network cannot distinguish "goal is to the right, path is clear" from "goal is to the right, trap is blocking".
2	Reward function	The uniform step penalty (-0.1) makes detours costly relative to the horizon. No progress-shaping term rewards getting closer.
3	Training distribution	Random trap placement rarely produces the blocked-path topology; the agent has seen too few detour-requiring situations.
4	Network capacity	Not a bottleneck at this scale.
5	PPO hyperparameters	Gradient balance is healthy; not the limiting factor.

9 Planned Improvements

9.1 State Representation: Direction-Quality Encoding

Replace the binary local encoding with a continuous *progress score* per direction:

$$\phi_{\text{dir}}(s) = \begin{cases} +2.0 & \text{wall} \\ -2.0 & \text{trap} \\ +2.0 & \text{goal} \\ \dfrac{d(s, g^*) - d(\text{nb}_{\text{dir}}, g^*)}{d_{\max}} & \text{otherwise} \end{cases}$$

Table 8: Bottleneck ranking

Rank	Factor	Explanation
1	State representation	ϕ_7, ϕ_8 (dx, dy) point toward the goal without encoding path quality per direction. The network cannot distinguish "goal is to the right, path is clear" from "goal is to the right, trap is blocking".
2	Reward function	The uniform step penalty (-0.1) makes detours costly relative to the horizon. No progress-shaping term rewards getting closer.
3	Training distribution	Random trap placement rarely produces the blocked-path topology; the agent has seen too few detour-requiring situations.
4	Network capacity	Not a bottleneck at this scale.
5	PPO hyperparameters	Gradient balance is healthy; not the limiting factor.

A positive score means moving in that direction reduces the distance to the goal; negative means it increases it. This gives the network a per-direction optimism signal without requiring global map knowledge.

9.2 Reward Shaping: Progress Bonus

Add a dense shaping term to reduce the temporal credit-assignment gap:

$$r_t \leftarrow r_t + \beta \cdot \frac{d(s_t, g^*) - d(s_{t+1}, g^*)}{d_{\max}}, \qquad \beta = 0.3$$

This makes the reward signal immediately informative: every step that gets closer to the goal yields a positive bonus, making detour costs visible to the agent in real time.

9.3 Training Distribution: Blocked-Path Curriculum

Introduce a specialised randomisation function that deliberately places traps on the shortest Manhattan path, forcing the agent to encounter detour-requiring situations during training.

9.4 Multi-Config Environment Generator

Replace the single-configuration `training_set_generator` with a composable generator that accepts multiple difficulty configurations and returns a shuffled flat list:

Listing 7: Multi-config environment generation

```
configs = [
  {board: small, n_goals: 1, n_traps: 0,  n_envs: 20},  # easy
  {board: small, n_goals: 1, n_traps: 2,  n_envs: 30},  #
     medium
  {board: large, n_goals: 3, n_traps: 10, n_envs: 50},  # hard
  {board: large, n_goals: 1, n_traps: 6,  n_envs: 40,   #
     blocked paths
   randomize_fn: randomize_board_blocked},
]
all_envs = build_training_set(configs, shuffle=True)
```

10 Summary

Where we stand

- A complete, modular PPO framework has been implemented from scratch and validated on two grid sizes.
- The large-grid agent (`agent_carter`) achieves **73% average success** across 10 test environments, with perfect 10/10 success on 5 of them.
- The gradient balance diagnostic confirms healthy actor/critic learning (ratio ≈ 0.8–1.0).
- The primary bottleneck is a capability gap — not overfitting — caused by the absence of a path-quality signal in the state representation and a step-penalty reward that discourages detours.
- Three targeted improvements (direction-quality encoding, progress shaping, blocked-path curriculum) are expected to raise the success rate to $\geq 85\%$.
- The LLM reward compiler, the thesis core contribution, is the next implementation milestone.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, *Proximal Policy Optimization Algorithms*, arXiv:1707.06347, 2017.

[2] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, *High-Dimensional Continuous Control Using Generalized Advantage Estimation*, ICLR 2016.

[3] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, *Stable-Baselines3: Reliable Reinforcement Learning Implementations*, JMLR 22(268):1–8, 2021.